PYTHON CLI QUIZ APPLICATION

Project Report

Project Type	Command-line quiz application with optional web deployment
Primary Technologies	Python, JSON
Optional Hosting Stack	Flask, Gunicorn, Render
Core Deliverables	12 questions, answer validation, scoring, result summary, tests

Prepared as a submission-ready starter report. Add your name, institution, and deployment links before final submission.

1. Abstract

The Python CLI Quiz Application is an interactive multiple-choice program designed to assess foundational Python knowledge. It loads question data from a separate JSON file, displays randomized questions, accepts and validates answers, provides immediate feedback, calculates a score and percentage, and displays a final performance message. An optional Flask browser interface is included so the same project can be demonstrated through a hosted Render URL while preserving the required command-line application.

2. Objectives

- Practice dictionaries, lists, loops, functions, and application flow.
- Keep question content separate from program logic using JSON.
- Handle invalid user input without crashing the quiz.
- Calculate scores and percentages accurately.
- Prepare a GitHub-ready project and an optional Render deployment.

3. Tools and Technologies

- Python: core quiz logic, input handling, loops, functions, and score calculation.
- JSON: external storage for questions, options, answers, and explanations.
- Flask: optional web routes and browser-based quiz interface.
- Gunicorn: production application server for Render.
- Git/GitHub: version control and public source repository.
- Render: deployment platform for the optional web version.

4. Functional Features

- 12 multiple-choice questions are provided in the question bank.
- Question order is randomized in each run.
- Each question presents four options labelled A through D.
- CLI input is normalized to uppercase and invalid choices are rejected with a retry prompt.
- The program reports whether the selected answer is correct and shows the correct option and explanation.
- The final summary includes raw score, percentage, and a performance message.
- Unit tests cover correct answers, invalid-input retry behavior, and score calculation.
- The optional Flask version provides a start screen, question screens, and a results screen.

5. System Design

The CLI entry point is main.py. It loads data from quiz_app/questions.json through load_questions(), presents questions with ask_question(), and controls the quiz flow with run_quiz(). The JSON file contains the question text, four options, the correct option letter, and an explanation. This separation allows new questions to be added without rewriting the core application. The optional app.py module reads the same JSON question bank and renders HTML templates for browser use.

6. How to Run

On Windows PowerShell, extract the project ZIP and enter the extracted folder. Create and activate a virtual environment with `py -m venv .venv` and `.\.venv\Scripts\Activate.ps1`. Run the command-line application using `python main.py`. Run automated tests with `python -m unittest discover -s tests -v`. To run the optional web interface locally, install dependencies using `python -m pip install -r requirements.txt` and start it with `python app.py`, then open http://127.0.0.1:5000.

7. Deployment and Submission

Push the project to a public GitHub repository. For Render, create a Web Service connected to that repository, use `pip install -r requirements.txt` as the build command and `gunicorn app:app` as the start command. Set SECRET_KEY as an environment variable; the included render.yaml requests a generated value. After deployment, test the public URL and add both GitHub and Render URLs to the README. The CLI application is run in a terminal; Render hosts the included optional web interface rather than providing an interactive CLI session.

8. Testing

The included unittest suite verifies that a correct answer is marked correctly, invalid input is rejected before accepting a valid answer, and score/percentage summaries are calculated correctly. Before submission, also perform a manual end-to-end run of the CLI and web version. Confirm that the question bank has at least ten questions and that the deployed Render URL loads successfully.

9. Conclusion

This project demonstrates core Python programming concepts in a practical quiz application. Its modular structure, external JSON data, input validation, scoring logic, automated tests, and optional deployable web interface make it suitable for a beginner portfolio project. The final submission should include the public GitHub repository, the working Render URL if deployed, and a LinkedIn post describing the completed work.

Before submission: personalize this report with your name and details, run the tests on your own machine, and replace any placeholder repository/deployment links with the actual URLs.